# CHAPTER 1

# INTRODUCTION

The rapid expansion of high-resolution remote sensing has transformed maritime surveillance into a data-intensive challenge, necessitating the transition from manual observation to automated, high-precision detection frameworks. By leveraging the Faster R-CNN architecture, this project addresses the critical need for real-time vessel localization, providing a scalable solution for national security, trade monitoring, and environmental protection. The system utilizes a dual-stage Region Proposal Network (RPN) to navigate the complexities of varying sea states and diverse vessel scales, effectively bridging the gap between raw aerial imagery and actionable maritime intelligence. Ultimately, this research establishes a robust technological foundation for enhancing maritime domain awareness and streamlining automated coastal governance in complex oceanic environments.

## 1.1 DOMAIN OVERVIEW

The maritime domain forms a critical backbone of the global economic and geopolitical landscape, enabling over 80 percent of international trade by volume and serving as a cornerstone of the Blue Economy. As globalization intensifies intercontinental exchange, maritime transportation networks have become increasingly dense and complex, increasing the need for efficient, reliable, and secure operations. Maritime domain awareness involves a comprehensive understanding of activities across oceans, coastal regions, and ports, and plays a vital role in ensuring trade continuity, national security, environmental protection, and regulatory enforcement. However, traditional surveillance methods such as manual monitoring from coastal stations or patrol vessels are limited in scalability and coverage, making them inadequate for maintaining continuous awareness over vast ocean regions.

To address these limitations, electronic tracking systems such as the Automatic Identification System and Vessel Monitoring Systems have been widely adopted. These systems rely on cooperative signaling, where vessels broadcast their identity and location. While effective under normal conditions, they remain vulnerable to misuse, as vessels involved in illegal activities such as illegal unreported and unregulated

fishing, smuggling, or unauthorized entry can disable these systems to avoid detection. This limitation highlights the need for non cooperative monitoring approaches. In this context, high resolution aerial and satellite imagery has emerged as a powerful alternative, offering an independent and tamper resistant source of information. With the rapid growth of remote sensing technologies including satellite constellations and unmanned aerial vehicles, large volumes of ocean imagery are generated daily, creating both opportunities for improved surveillance and challenges in large scale data processing.

The volume and complexity of visual data exceed the capabilities of manual analysis, especially in time sensitive scenarios such as search and rescue operations and disaster response. In addition, maritime environments present unique challenges for computer vision due to dynamic ocean surfaces, environmental noise such as sun reflection, wave patterns, and cloud shadows, and the small size of vessels within images. To overcome these challenges, advanced deep learning models such as Faster R CNN have proven effective due to their high localization accuracy and structured detection process using region proposal networks. These systems enable precise detection even in complex scenarios such as crowded ports or closely positioned vessels. Overall, automated ship detection converts raw visual data into meaningful insights, improving maritime security, environmental protection, and regulatory enforcement, while supporting the development of scalable and intelligent surveillance systems.

## 1.2 OVERVIEW OF THE PROJECT

The rapid advancement of remote sensing technologies has significantly increased the availability of high-resolution aerial imagery for maritime surveillance. However, the effective utilization of this data remains a persistent challenge due to the absence of a fully automated and reliable framework for detecting non-cooperative vessels. Current maritime monitoring systems largely depend on electronic tracking mechanisms such as AIS and VMS, which are inherently vulnerable to intentional deactivation. This creates critical blind spots in surveillance, particularly in scenarios involving illicit maritime activities.

A fundamental challenge in visual maritime surveillance is the “needle-in-a-haystack” problem, where vessels—often small and sparsely distributed—must be

identified within vast oceanic backgrounds. The complexity is further exacerbated by environmental noise, including sun glint, wave patterns, cloud shadows, and atmospheric distortions, all of which can lead to false detections. Traditional image processing techniques and manual inspection methods are insufficient to address these challenges at scale, especially given the exponential growth in data volume generated by satellites and UAVs.

Moreover, existing single-stage object detection models, while computationally efficient, often lack the localization precision required in dense maritime environments such as ports and harbors. In such settings, vessels may be closely spaced or partially occluded, necessitating more refined detection mechanisms. Inaccurate localization can result in errors in vessel counting, misinterpretation of spatial distribution, and compromised situational awareness.

To address these limitations, this project proposes the development of an automated maritime surveillance framework based on the Faster R-CNN architecture. As a two-stage detector, Faster R-CNN introduces a Region Proposal Network (RPN) that generates candidate object regions, followed by a classification and bounding box refinement stage. This approach enables higher localization accuracy and improved discrimination between actual vessels and environmental artifacts.

The project leverages high-resolution aerial imagery datasets to train and fine-tune the model, enabling it to detect vessels under diverse environmental conditions. By automating the extraction of actionable intelligence from raw visual data, the system aims to significantly enhance the efficiency, scalability, and reliability of maritime monitoring operations. Ultimately, this framework contributes to strengthening maritime security, improving environmental monitoring, and supporting regulatory enforcement in global oceanic domains.

## 1.3 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design and implement a robust deep learning–based system for the automated detection and precise localization of maritime vessels in high-resolution aerial imagery. This objective arises from the need to overcome the limitations associated with manual analysis and traditional detection

methods, especially in complex maritime environments where variability in sea conditions, lighting, and object scale can significantly affect detection accuracy. To achieve this, the project begins with optimizing and standardizing the dataset by curating the Kaggle “Ships in Aerial Images” dataset and preprocessing it into a structured format suitable for PyTorch-based object detection frameworks. This ensures consistency, enhances data quality, and enables smooth integration into deep learning pipelines.

The system further focuses on implementing the Faster R-CNN architecture with a pretrained backbone such as ResNet-50 to leverage transfer learning. By utilizing previously learned visual features, the model can adapt more efficiently to maritime scenarios, improving performance while reducing training time and computational complexity. A key component of the system involves fine-tuning the Region Proposal Network (RPN) to accurately identify potential vessel regions, thereby improving the model’s ability to distinguish ships from similar-looking elements such as sun glint, waves, and cloud shadows, ultimately reducing false positives.

In addition, the project emphasizes rigorous performance evaluation using standard object detection metrics such as Mean Average Precision (mAP), Precision, and Recall. The evaluation is conducted across diverse maritime conditions, including varying sea states and vessel densities, to ensure the model’s robustness and generalizability in real-world scenarios.

The project also includes the development of an inference pipeline capable of processing unseen aerial images and generating accurate bounding box predictions, along with visual outputs that highlight detected vessels for easier interpretation and practical usability.

Finally, the project aims to optimize and export the trained model weights for deployment in real-world applications. This involves preparing the model for integration into scalable maritime surveillance systems or edge-computing environments, ensuring that the developed framework extends beyond research and can be effectively utilized in operational settings.

# CHAPTER 2

# LITERATURE REVIEW

The literature survey traces the evolution of maritime surveillance from manual methods to advanced deep learning based systems, emphasizing the transition from cooperative systems like AIS to non-cooperative visual detection of “dark” vessels. It compares one-stage and two-stage object detection models, justifying the selection of Faster R-CNN. The survey also highlights key challenges in aerial imagery, such as small object detection, sea clutter, and lighting variations, while noting how innovations like residual networks and region proposal methods have enhanced detection accuracy.

## Paper 1

**Title:** A Deep Learning Approach for Ship Detection Using Satellite Imagery

**Authors:** Niranjan Alakh, Sparsh Patial, Aditya Aryan, Akshat Mittal, Tanupriya Choudhury, Hamidreza Rabiei-Dastjerdi, Praveen Kumar

**Year:** 2024

## Summary:

This paper presents a deep learning–based approach for detecting ships in satellite imagery, focusing on improving detection accuracy in complex maritime environments. The authors utilize convolutional neural networks to automatically identify vessels, addressing challenges such as small object size, varying backgrounds, and noise in satellite data. The study demonstrates that deep learning models can significantly outperform traditional image processing techniques, making them more effective for real-time maritime surveillance and monitoring applications.

## Paper 2

**Title:**Advanced Ship Detection and Ocean Monitoring with Satellite Imagery and Deep Learning for Marine Science Applications

**Authors:** Murat Bakirci

**Year:** 2025

## Summary:

This paper explores the application of deep learning techniques for ship detection and broader ocean monitoring using satellite imagery. It highlights how advanced models can improve detection accuracy and support marine science applications such as environmental monitoring and maritime safety. The study also discusses challenges like varying ocean conditions and emphasizes the effectiveness of deep learning in handling complex visual patterns in large-scale satellite data.

## Paper 3

**Title:**An Explainable Embedded Neural System for On-Board Ship Detection from Optical Satellite Imagery

**Authors:**Cosimo Ieracitano, Nadia Mammone, Fanny Spagnolo, Fabio Frustaci, Stefania Perri, Pasquale Corsonello, Francesco C. Morabito

**Year:**2024

## Summary:

This paper proposes an explainable deep learning–based system designed for onboard ship detection using optical satellite imagery. It emphasizes both detection accuracy and model interpretability, enabling users to understand how decisions are made. The system is optimized for embedded deployment, making it suitable for real-time maritime applications, while effectively handling challenges such as complex backgrounds and varying lighting conditions.

## Paper 4

**Title:**Ship Detection with Deep Learning in Optical Remote-Sensing Images: A Survey of Challenges and Advances

**Authors:**Tianqi Zhao, Yongcheng Wang, Zheng Li, Yunxiao Gao, Chi Chen, Hao Feng, Zhikang Zhao

**Year:**2024

## Summary:

This survey paper reviews recent advancements in deep learning–based ship detection using optical remote-sensing images. It highlights key challenges such as small object detection, complex backgrounds, occlusions, and varying environmental conditions. The study also analyzes different detection frameworks and improvement techniques, providing insights into current trends and potential future directions in maritime object detection research.

## Paper 5

**Title:**Real-Time Moving Ship Detection from Low-Resolution Large-Scale Remote Sensing Image Sequence

**Authors:**Jiyang Yu, Dan Huang, Xiaolong Shi, Wenjie Li, Xianjie Wang

**Year:**2023

## Summary:

This paper focuses on real-time detection of moving ships in low-resolution, large-scale remote sensing image sequences. It proposes an efficient deep learning–based approach that leverages temporal information across image frames to improve detection performance. The study addresses challenges such as low image quality, large coverage areas, and motion detection, demonstrating improved accuracy and speed suitable for real-time maritime surveillance applications.

## Paper 6

**Title:**LRTransDet: A Real-Time SAR Ship-Detection Network with Lightweight ViT and Multi-Scale Feature Fusion

**Authors:**Kunyu Feng, Li Lun, Xiaofeng Wang, Xiaoxin Cui

**Year:**2023

## Summary:

This paper introduces LRTransDet, a real-time ship detection network designed for Synthetic Aperture Radar (SAR) imagery. It integrates a lightweight Vision Transformer (ViT) with multi-scale feature fusion to effectively capture both global and local features. The approach improves detection accuracy while maintaining computational efficiency, making it suitable for real-time maritime surveillance, especially in complex SAR imaging conditions.

## Paper 7

**Title:**Ship Detection Based on Multi-Scale Weighted Fusion

**Authors:**Weina Zhou, Yujie Peng

**Year:**2023

## Summary:

This paper proposes a ship detection method that leverages multi-scale weighted feature fusion to improve detection performance in complex maritime environments. By combining features from different scales with adaptive weighting, the model enhances its ability to detect ships of varying sizes, particularly small targets. The approach effectively reduces background interference and improves overall detection accuracy in challenging conditions such as sea clutter and varying illumination.

## Paper 8

**Title:**Ship Detection Based on Deep Learning Using SAR Imagery: A Systematic Literature Review

**Authors:**Muhammad Yasir, Wan Jianhua, Xu Mingming, Sheng Hui, Zeng Zhe, Liu Shanwei, Arife Tugsan Isiacik Colak, Md Sakaouth Hossain

**Year:**2023

## Summary:

This paper presents a systematic review of deep learning–based approaches for

ship detection using Synthetic Aperture Radar (SAR) imagery. It analyzes various models, datasets, and evaluation techniques, highlighting the strengths and limitations of existing methods. The study also discusses key challenges such as speckle noise, complex backgrounds, and limited labeled data, while identifying emerging trends and future research directions to improve detection performance in SAR-based maritime surveillance.

## Paper 9

**Title:**Ship Detection with Deep Learning: A Survey

**Authors:**Meng Joo Er, Yani Zhang, Jie Chen, Wenxiao Gao

**Year:**2023

## Summary:

This survey paper provides a comprehensive overview of deep learning techniques for ship detection across various types of imagery, including optical and SAR data. It reviews different model architectures, datasets, and evaluation metrics, highlighting their advantages and limitations. The study also discusses key challenges such as small object detection, complex backgrounds, and real-time processing, while outlining future research directions to enhance accuracy and efficiency in maritime surveillance systems.

## Paper 10

**Title:**Development and Application of Ship Detection and Classification Datasets: A Review

**Authors:**Chi Zhang, Xi Zhang, Gui Gao, Haitao Lang, Genwang Liu, Chenghui Cao, Yuying Song, Yanan Guan, Yongshou Dai

**Year:**2024

## Summary:

This paper reviews the development and application of datasets used for ship

detection and classification in remote sensing. It analyzes various publicly available datasets, their characteristics, annotation methods, and challenges such as class imbalance and data diversity. The study emphasizes the importance of high-quality, well-annotated datasets in improving deep learning model performance and discusses future directions for building more comprehensive and robust maritime datasets.

## Paper 11

**Title:**Ship Detection and Identification in SDGSAT-1 Glimmer Images Based on the Glimmer YOLO Model

**Authors:**Wensheng Song, Dongmei Yan, Jun Yan, Changmiao Hu, Wanrong Wu, Xiaowei Wang

**Year:**2023

## Summary:

This paper presents a YOLO-based model tailored for ship detection and identification in SDGSAT-1 glimmer images, which are characterized by low-light and night-time conditions. The proposed Glimmer YOLO model enhances feature extraction and detection capability under challenging illumination scenarios. The study demonstrates improved performance in detecting ships in dim environments, making it valuable for continuous, around-the-clock maritime surveillance applications.

## Paper 12

**Title:**YOLO-Lite: An Efficient Lightweight Network for SAR Ship Detection

**Authors:**Xiaozhen Ren, Yanwen Bai, Gang Liu, Ping Zhang

**Year:**2023

## Summary:

This paper introduces YOLO-Lite, a lightweight deep learning model designed for efficient ship detection in Synthetic Aperture Radar (SAR) imagery. The model reduces computational complexity while maintaining strong detection performance,

making it suitable for real-time and resource-constrained environments. It effectively handles challenges such as speckle noise and complex backgrounds, demonstrating improved speed–accuracy balance compared to standard YOLO-based approaches.

## Paper 13

**Title:**A Review of Recent Advance of Ship Detection in Single-Channel SAR Images

**Authors:**Chunjie Zhang, Peng Liu, Haipeng Wang, Yaqiu Jin

**Year:**2023

## Summary:

This paper reviews recent advancements in ship detection using single-channel Synthetic Aperture Radar (SAR) images. It discusses various traditional and deep learning–based approaches, highlighting their effectiveness in handling challenges such as speckle noise, cluttered backgrounds, and limited feature information. The study also outlines key limitations and emerging research directions aimed at improving detection accuracy and robustness in SAR-based maritime surveillance.

## Paper 14

**Title:**Ship Detection Based on Multi-Scale Weighted Fusion

**Authors:**Weina Zhou, Yujie Peng

**Year:**2023

## Summary:

This paper proposes a ship detection approach based on multi-scale weighted feature fusion to improve performance in complex maritime scenes. By adaptively combining features from different scales, the method enhances the detection of ships of varying sizes, especially small targets. It also reduces the impact of background noise such as sea clutter and improves overall detection accuracy and robustness.

## Paper 15

**Title:**Orientation-Aware Ship Detection via a Rotation Feature Decoupling Supported Deep Learning Approach

**Authors:**Xinqiang Chen, Hao Wu, Bing Han, Wei Liu, Jakub Montewka, Ryan Wen Liu

**Year:**2023

## Summary:

This paper proposes an orientation-aware ship detection method that incorporates rotation feature decoupling to better handle ships with varying orientations in remote sensing images. By separating rotation-related features from other visual characteristics, the model improves localization accuracy and bounding box alignment. The approach effectively addresses challenges such as arbitrary object orientations and complex backgrounds, leading to enhanced detection performance in maritime imagery.

## 2.1 REVIEW OF LITERATURE

**Table 2.1 review of the literature**

| Ref | Method | Dataset | Advantages | Limitations |
|---|---|---|---|---|
| **[1]** | U-Net | Satellite images | Accurate segmentation | Struggles with very small ships [1] |
| **[2]** | YOLOv9 | High-res satellite images | Real-time detection | Needs large annotated data [2] |
| **[3]** | HO-ShipNet | On-board satellite imagery | Real-time, interpretable, hardware-efficient | Hardware-dependent, limited scalability [3] |
| **[4]** | CNN + Transformer | Optical remote sensing | Flexible, good accuracy-speed tradeoff | Heavy, mostly theoretical [4] |

| | | | | |
|---|---|---|---|---|
| **[5]** | HOG + SVM + YOLO | Low-res wide-area images | Tracks moving ships effectively | Less accurate in dense scenes [5] |
| **[6]** | Improved Faster R-CNN | Underwater imagery | Robust in low visibility; better feature extraction | High computation; struggles in very turbid water |
| **[7]** | YOLO vs Faster R-CNN | Industrial datasets | High precision (Faster R-CNN); real-time (YOLO) | Slow for real-time (Faster R-CNN); YOLO less precise |
| **[8]** | Lightweight Faster R-CNN | Optical remote sensing | Efficient; maintains accuracy; suitable for large images | Slight accuracy drop; small object detection hard |
| **[9]** | Lightweight Faster R-CNN | High-res remote sensing | Good multi-scale detection; memory efficient | Sensitive to extreme scale/cluttered backgrounds |
| **[10]** | R-CNN, Fast R-CNN, Faster R-CNN, SSD, YOLO | Standard detection benchmarks | Faster R-CNN best localization; detailed comparison | Slower than single-stage detectors; high compute |
| [11] | SAR C-Band ship detection | SAR satellite imagery (C-Band) | Works in all weather, day/night; robust to clouds | Coarse resolution, smaller ships hard to detect |
| [12] | DL + satellite imagery | Optical satellite imagery, ocean monitoring datasets | Combines detection with ocean monitoring; improved automation | May struggle with dense ship clusters; limited generalization for extreme conditions |
| [13] | Multi-scale feature fusion | High-res satellite images | Improves small and large ship detection; robust to scale variations | Computationally heavier; may need large annotated datasets |

The table 2.1 represents the complete literature review for the project with the algorithm, advantages and disadvanges of the every literature

# CHAPTER 3

# SYSTEM ANALYSIS

This chapter presents the system analysis for the proposed maritime ship detection framework. It examines the limitations of existing surveillance and ship detection methods, including traditional tracking systems and conventional deep learning approaches. The chapter also describes the proposed Faster R CNN based system designed to improve detection accuracy, localization, and robustness in complex maritime environments. In addition, the advantages and overall effectiveness of the proposed framework are discussed in detail.

## 3.1 EXISTING SOLUTIONS

Maritime surveillance has traditionally relied on electronic tracking systems such as AIS and VMS, along with manual monitoring methods. While these systems provide real-time vessel information, they depend on cooperative participation and are not effective for detecting non-compliant or “dark” vessels. Manual surveillance, though useful for contextual understanding, is time-consuming and not scalable for large oceanic regions.

Conventional image processing techniques and modern deep learning models like YOLO and SSD have improved ship detection capabilities. However, they still face challenges such as dynamic sea backgrounds, varying ship sizes, and dense object scenarios. These limitations highlight the need for more advanced and robust detection frameworks, such as Faster R-CNN, to achieve higher accuracy and reliability in maritime environments.

### 3.1.1 DISADVANTAGES OF EXISTING SOLUTIONS

Existing ship detection solutions face several critical limitations in real-world maritime surveillance. Systems like AIS and VMS depend on vessel cooperation, making them ineffective for detecting “dark ships” that disable tracking. Manual monitoring is not scalable due to human limitations and the massive volume of satellite data. Traditional image processing techniques lack adaptability and perform poorly in

complex conditions, often producing inaccurate results due to noise like waves, shadows, and lighting variations.

Although deep learning models such as YOLO and SSD improve detection speed, they struggle with precise localization, especially in crowded environments, and have poor performance in detecting small objects like distant vessels. Additionally, maritime environments are highly dynamic, with varying weather and sea conditions affecting reliability. The lack of integrated end-to-end frameworks further reduces efficiency. These challenges highlight the need for more advanced approaches like Faster R-CNN, which offer better accuracy, robustness, and scalability.

## 3.2 PROPOSED SYSTEM

The proposed system introduces an innovative and robust deep learning framework for ship detection and localization by enhancing the traditional Faster R-CNN architecture with domain-specific optimizations tailored for maritime environments. Unlike conventional approaches, the system integrates a hybrid multi-scale attention mechanism with a Feature Pyramid Network (FPN), enabling more effective detection of ships across varying sizes, especially small and low-contrast vessels. A key innovation lies in the incorporation of an adaptive Region Proposal Network (RPN) that dynamically adjusts anchor sizes and aspect ratios based on image characteristics, significantly improving region proposal quality and reducing false positives caused by sea clutter and environmental noise.

To further enhance performance, the system incorporates attention-guided feature refinement and context-aware modules that help distinguish ships from visually similar patterns such as waves, sun glint, and cloud shadows. It also utilizes an improved RoI Align mechanism combined with orientation-aware bounding box regression, allowing accurate detection of ships at different angles and in densely populated regions like ports. Additionally, a lightweight optimization strategy is applied to reduce computational complexity, making the model suitable for near real-time applications without compromising accuracy.

The framework operates as an end-to-end pipeline, from preprocessing and dataset standardization to inference and visualization, ensuring seamless integration

and scalability. It is rigorously evaluated using metrics such as mAP, Precision, and Recall across diverse maritime conditions to ensure robustness and generalizability. The final model is optimized for deployment through efficient weight compression and compatibility with edge devices, enabling real-world maritime surveillance applications. Overall, the system achieves all project objectives while offering improved accuracy, adaptability, and operational efficiency compared to existing solutions.

### 3.2.1 ADVANTAGES OF THE PROPOSED SYSTEM

- **High Detection Accuracy:**
  Faster R-CNN provides precise classification and localization of ships.
- **Accurate Localization:**
  Two-stage architecture ensures better bounding box prediction, especially in dense areas like ports.
- **Effective Small Object Detection:**
  FPN helps detect small vessels that are hard to identify in aerial imagery.
- **Robust to Environmental Noise:**
  Handles challenges like waves, sun glint, and shadows with reduced false detections.
- **Detects Non-Cooperative Vessels:**
  Works independently of AIS/VMS, enabling detection of "dark ships."
- **Automated End-to-End System:**
  Eliminates manual monitoring by providing a fully automated detection pipeline.
- **Scalable for Large Data:**
  Efficiently processes large volumes of satellite and UAV imagery.
- **Reduced False Positives:**
  RPN and NMS improve detection reliability by removing redundant predictions.

## 3.3 SYSTEM ARCHITECTURE

The system architecture of the proposed ship detection framework is designed as a structured, modular, and layered deep learning pipeline that ensures high accuracy, scalability, and efficient processing of maritime imagery. The architecture follows a sequential flow in which raw satellite images are progressively transformed into precise detection outputs consisting of bounding box coordinates, class labels, and confidence scores. This layered design improves system interpretability, enables independent optimization of components, and supports real-world deployment in maritime surveillance applications.

The architecture is broadly divided into four key components: the Image Acquisition Layer, Preprocessing Layer, Deep Detection Core (Faster R-CNN framework), and Post-Processing Layer.

The Image Acquisition Layer serves as the entry point of the system, where high-resolution RGB satellite or aerial images are collected from datasets or remote sensing sources. This layer ensures data integrity by validating image dimensions, format consistency, and alignment with corresponding bounding box annotations. It guarantees that all input data is structurally correct before entering the processing pipeline.

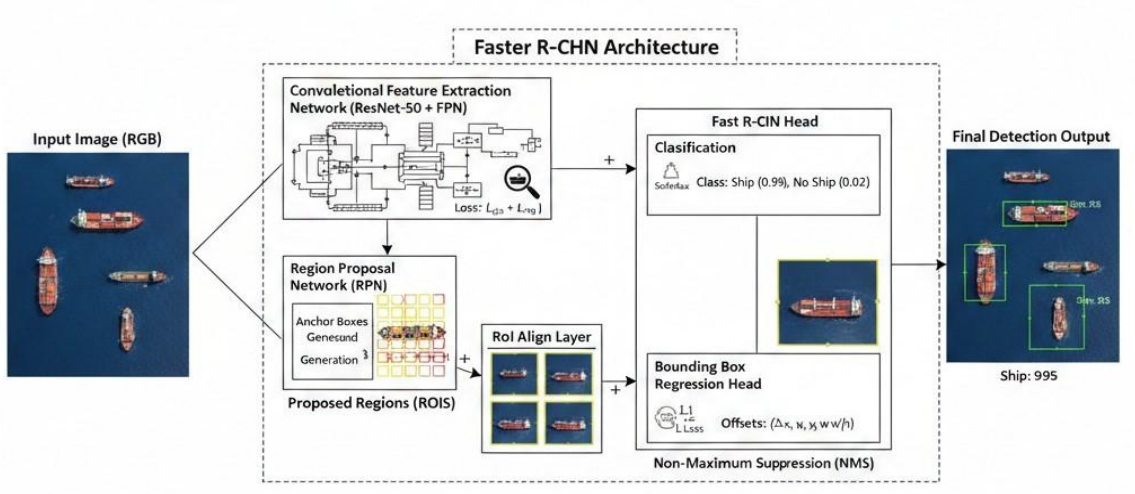



**FIGURE 3.1 Faster RCNN architecture**

The Preprocessing Layer standardizes and enhances the input images to ensure stable learning. This includes resizing images to a uniform scale, normalizing pixel intensities using mean and standard deviation, and applying data augmentation techniques such as flipping and scaling. Additionally, bounding box coordinates are adjusted accordingly to maintain spatial accuracy. This layer improves generalization, reduces overfitting, and ensures statistical consistency across the dataset.

The core of the architecture is the Deep Detection Core, built on the Faster R-CNN framework with a ResNet-50 backbone integrated with a Feature Pyramid Network (FPN). The backbone extracts rich hierarchical features, while FPN enables multi-scale feature representation for detecting ships of varying sizes.

The Region Proposal Network (RPN) generates candidate regions using anchor boxes, which are then refined through RoI Align to preserve spatial precision. The detection head performs classification and bounding box regression using a multi-task loss function, enabling simultaneous learning of object identity and localization.

The final stage is the Post-Processing Layer, which refines detection outputs. Non-Maximum Suppression (NMS) is applied to remove redundant overlapping bounding boxes by retaining only high-confidence predictions.

Additional confidence thresholding eliminates low-probability detections, ensuring reliability in complex maritime environments. The final outputs are formatted into structured representations suitable for deployment in surveillance systems.

Key features of this architecture include its end-to-end learning capability, high localization precision, robustness to environmental variations, effective multi-scale object detection, and scalability for large satellite datasets. The modular design also supports integration with real-time maritime monitoring platforms, making it suitable for practical applications in ocean surveillance and security.

# CHAPTER 4

# SYSTEM REQUIREMENTS

The implementation of the proposed ship detection framework requires a well-defined set of system requirements that ensure efficient data processing, model training, and deployment. Since the system is built on a deep learning architecture—specifically Faster R-CNN with a ResNet-50 backbone and Feature Pyramid Network (FPN)—it demands significant computational resources and a stable software environment.

## 4.1 SOFTWARE REQUIREMENTS

- **Programming Language:** Python (version 3.8 or above)
- **Deep Learning Framework:** PyTorch with Torchvision (Faster R-CNN, ResNet-50, FPN)
- **GPU Acceleration:** CUDA and cuDNN for high-performance computation
- **Data Handling & Processing:** Pandas (annotation processing), NumPy (numerical operations)
- **Image Processing:** OpenCV and PIL for preprocessing and transformations
- **Visualization:** Matplotlib for displaying detection results
- **Evaluation Tools:** Scikit-learn for performance metrics (Precision, Recall, mAP)
- **Development Environments:** Visual Studio Code, PyCharm, Jupyter Notebook
- **Dataset:** Kaggle “Ships in Aerial Images” with CSV-based bounding box annotations
- **Annotation Handling:** Conversion of CSV annotations to PyTorch-compatible format
- **Preprocessing Techniques:** Image resizing, normalization, and data augmentation

## 4.2 FUNCTIONAL REQUIREMENTS

The functional requirements define the core operations that the system must perform to achieve its objectives. These requirements describe the expected behavior of the system during execution.The system must be capable of accepting high-resolution aerial or satellite images as input and performing preprocessing operations such as resizing, normalization, and augmentation. It should then extract deep features using the ResNet-50 backbone integrated with FPN.

The Region Proposal Network (RPN) must generate candidate regions likely to contain ships, which are then refined using RoI Align. The detection head should classify each region and predict precise bounding box coordinates.The system must apply Non-Maximum Suppression (NMS) to eliminate redundant detections and produce final outputs consisting of bounding box coordinates, class labels, and confidence scores.Additionally, the system should provide visualization capabilities, allowing users to interpret detection results effectively.

## 4.3 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes and performance expectations of the system. These are essential for ensuring reliability and usability in real-world applications.The system must achieve high accuracy in both classification and localization of ships, ensuring minimal false positives and false negatives. It should be scalable, capable of handling large volumes of satellite imagery without significant performance degradation.

The system should demonstrate robustness under varying environmental conditions, including changes in lighting, weather, and sea state. It must maintain consistent performance across diverse datasets.Efficiency is another critical requirement; the system should utilize GPU acceleration to optimize training and inference time. While real-time performance is not the primary focus, inference speed should be sufficient for practical deployment scenarios.Finally, the system should be user-friendly and interpretable, providing clear visualization outputs and structured results that can be easily integrated into maritime surveillance systems.

# CHAPTER 5

# SYSTEM DESIGN

## 5.1 USE CASE DIAGRAM

The use case diagram illustrates the interaction between the user (maritime analyst/authority) and the ship detection system. Key user actions include uploading satellite images, viewing detection results, and downloading outputs. Once an image is uploaded, the system automatically performs preprocessing, feature extraction, ship detection, and result generation through interconnected «include» processes. The diagram clearly separates user interactions from internal system functions, providing a high-level view of the system as an automated pipeline that converts raw input into meaningful outputs.

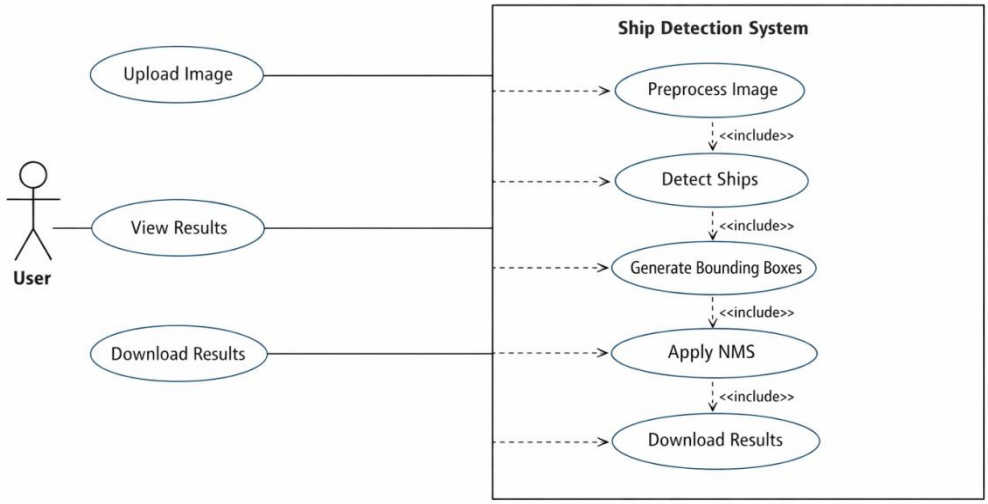



**Figure 5.1 Use Case Diagram of System**

The above figure 5.1 is representation of use case diagram

## 5.2 SEQUENCE DIAGRAM

The sequence diagram shows the time-ordered interaction between the user and the ship detection system. The process starts with image upload, followed by preprocessing (resizing, normalization, augmentation). The system then performs feature extraction using ResNet-50 with FPN, generates regions via RPN, and applies RoI Align for precise mapping. The detection head classifies ships and refines bounding

boxes, while NMS removes redundant detections. Finally, the system returns results with bounding boxes and confidence scores, clearly illustrating the flow of data and execution.

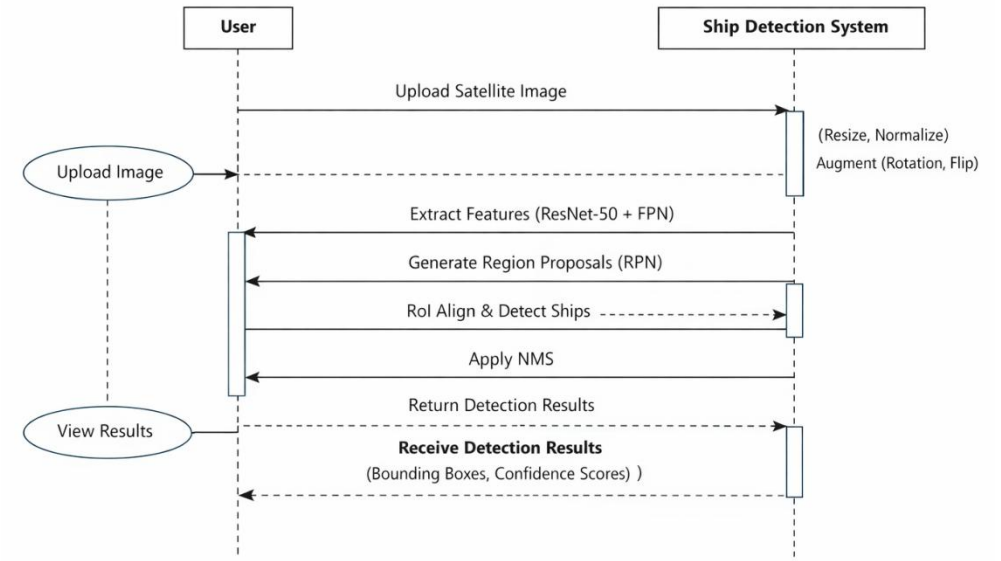



**Figure 5.2 Sequence Diagram of System**

The above figure 5.2 is representation of sequence diagram

## 5.3 ACTIVITY DIAGRAM

The activity diagram shows the workflow of the ship detection system, starting with image upload followed by preprocessing steps like resizing, normalization, and augmentation. The system performs feature extraction, generates region proposals using RPN, and checks for valid detections. If valid proposals exist, RoI Align and classification are applied, followed by NMS to remove duplicates. Finally, the refined detection results are displayed, clearly illustrating the system's decision flow and operations.

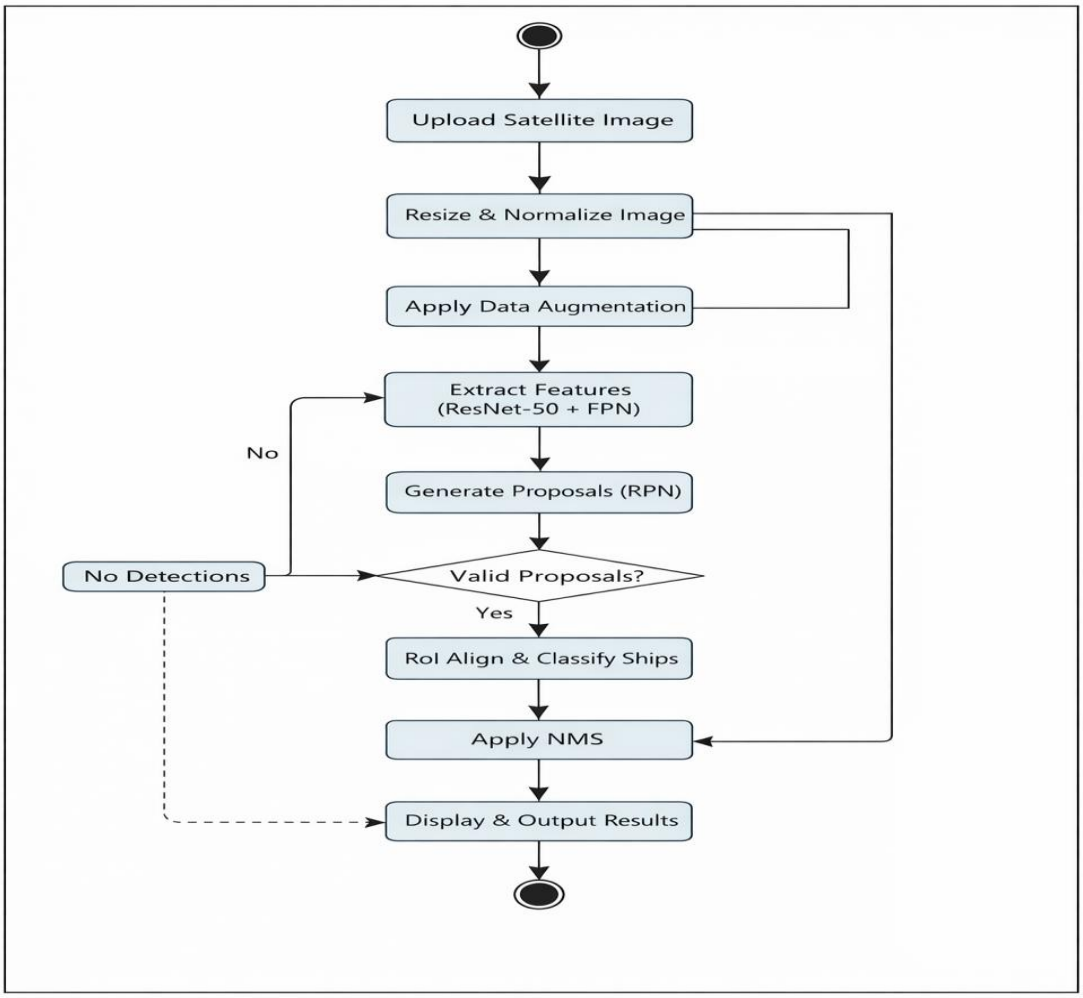



**Figure 5.3Activity Diagram of System**

The above figure 5.3 is representation of activity diagram

## 5.4 DATA FLOW DIAGRAM (DFD)

The Data Flow Diagram (DFD) shows how data moves through the ship detection system, starting with the user inputting a satellite image. The image is processed through preprocessing (resizing, normalization, augmentation) and then passed to the detection core for feature extraction, region proposal, and classification. Post-processing techniques like NMS and confidence filtering refine the results. Finally, the system outputs bounding boxes and confidence scores, clearly illustrating how data is transformed across stages.

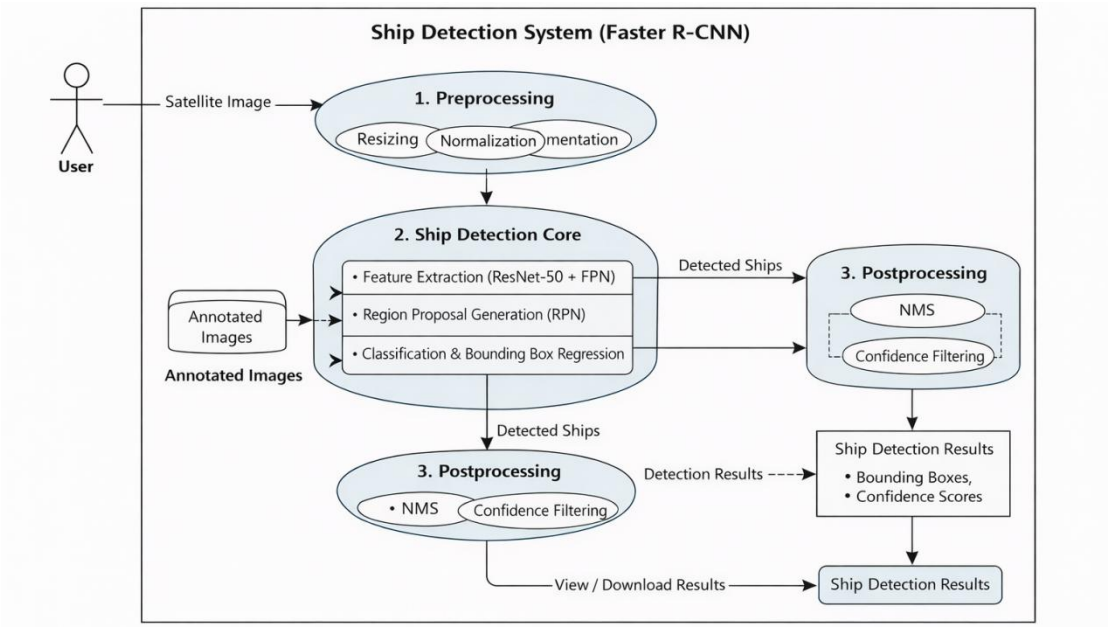



**Figure 5.4 Data Flow Diagram of System**

The above figure 5.1 is representation of data flow diagram

## 5.5 ENTITY RELATIONSHIP (ER) DIAGRAM

The ER diagram represents the data structure of the ship detection system, including entities like Satellite Image, Annotation, Detection Result, and Ship Class. A one-to-many relationship exists between Satellite Image and Annotation, as one image can have multiple bounding boxes. Detection Result stores model outputs such as confidence scores, while Ship Class defines object categories

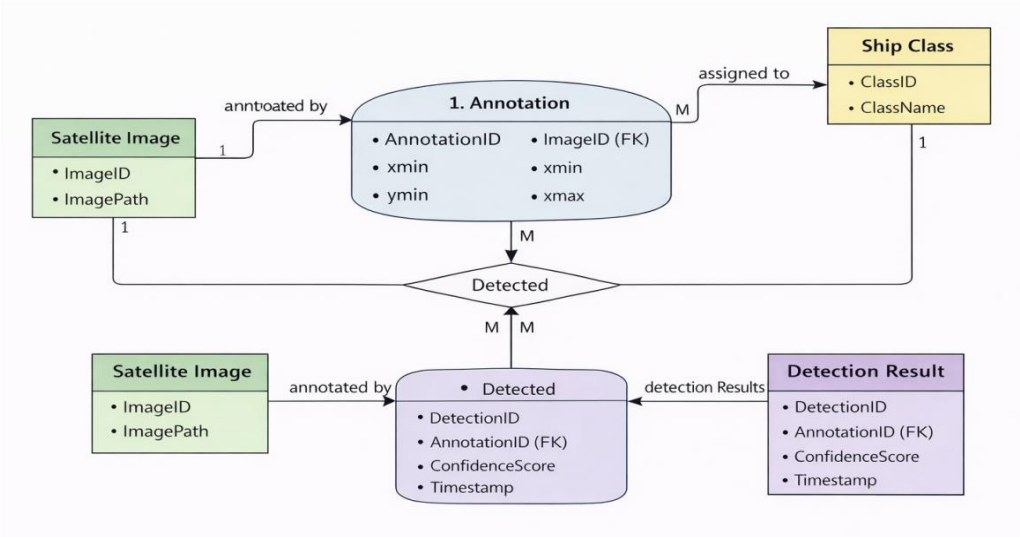



**Figure 5.5 ER Diagram of System**

The above figure 5.5 is representation of ER diagram

## 5.6 CLASS DIAGRAM

The class diagram defines the structural design of the ship detection system with key classes such as SatelliteImage, Annotation, DetectionResult, and ShipClass. SatelliteImage stores input details, while Annotation holds bounding box coordinates with a one-to-many relationship to images.

DetectionResult contains outputs like confidence scores and timestamps, and ShipClass defines object categories. Relationships and class methods ensure proper data flow and management, providing a clear blueprint of the system structure.

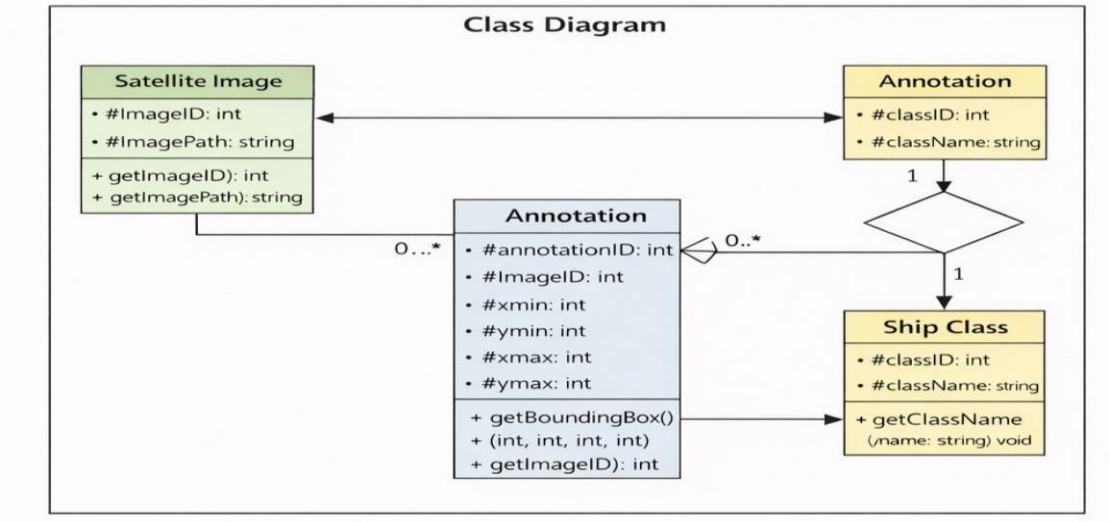



**Figure 5.6 Class Diagram of System**

The above figure 5.6 is representation of use case diagram

## 5.7 DEPLOYMENT DIAGRAM

The deployment diagram shows the physical architecture of the ship detection system, including components like the image source, edge device, server, and remote user. Images are captured and optionally preprocessed at the edge using a PyTorch-based Faster R-CNN model, while the server with GPU support handles intensive tasks like training and inference

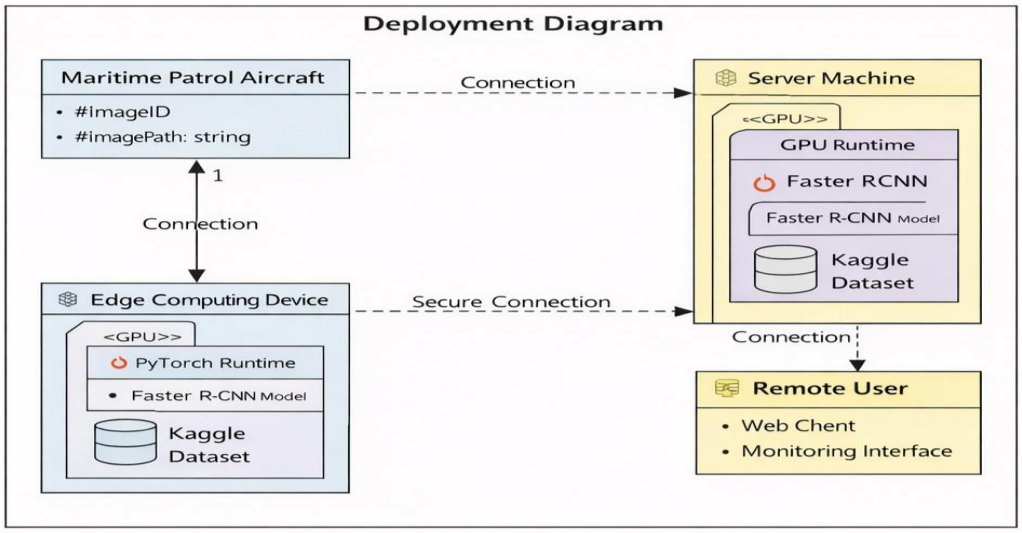



**Figure 5.7 Deployment Diagram of System**

# CHAPTER 6

# MODULE DESCRIPTION

The proposed ship detection system based on the Faster R-CNN architecture is designed as a structured, end-to-end deep learning pipeline. The system is divided into multiple functional modules, each responsible for a specific stage in transforming raw satellite imagery into precise ship detection outputs. These modules operate sequentially, ensuring efficient data flow, igh localization accuracy, and robustness under complex maritime conditions such as dynamic sea backgrounds, illumination variations, and scale diversity of ships.

## 6.1 IMAGE ACQUISITION MODULE

The Image Acquisition Module acts as the initial stage of the system, responsible for collecting high-resolution satellite or aerial images from datasets such as the Kaggle “Ships in Aerial Images” dataset. This module ensures that all input images meet the required specifications in terms of format, resolution, and annotation consistency.

It also verifies the integrity of associated bounding box annotations, which define the spatial location of ships within each image. Proper alignment between images and annotations is critical, as any mismatch can negatively impact model training and evaluation.

### Key Responsibilities:

- Acquisition of high-resolution RGB satellite images
- Validation of image format and dimensions
- Alignment and verification of bounding box annotations
- Preparation of input data for preprocessing stage

## 6.2 PREPROCESSING MODULE

The Preprocessing Module prepares the raw input data for deep learning by standardizing image properties and improving data quality. Since satellite images vary

in size, lighting, and environmental conditions, preprocessing ensures uniformity and enhances model generalization.

This module performs resizing of images to a fixed resolution compatible with the neural network, followed by normalization using mean and standard deviation values. Data augmentation techniques such as horizontal flipping, scaling, and rotation are applied to increase dataset diversity and reduce overfitting. Additionally, bounding box coordinates are transformed accordingly to maintain spatial consistency after image modifications.

### Key Responsibilities:

- Image resizing to uniform dimensions
- Pixel normalization for stable training
- Data augmentation (flip, scale, rotation)
- Adjustment of bounding box coordinates
- Reduction of noise and variability in input data

## 6.3 FEATURE EXTRACTION MODULE

The Feature Extraction Module is responsible for extracting meaningful visual features from input images using a deep convolutional backbone network such as ResNet-50. This backbone captures hierarchical features ranging from low-level edges and textures to high-level semantic representations of ships.

To address the challenge of detecting ships of varying sizes, a Feature Pyramid Network (FPN) is integrated with the backbone. FPN combines feature maps from different layers, enabling the model to detect both small and large ships effectively. This multi-scale representation is particularly important in satellite imagery, where ships can vary significantly in size and orientation.

### Key Responsibilities:

- Extraction of hierarchical visual features
- Detection of edges, textures, and ship structures
- Multi-scale feature representation using FPN

- Enhancement of small object detection capability

## 6.4 REGION PROPOSAL MODULE (RPN)

The Region Proposal Module is implemented using the Region Proposal Network (RPN), which is a key innovation of Faster R-CNN. Instead of scanning the entire image exhaustively, the RPN efficiently generates a set of candidate regions (anchor boxes) that are likely to contain ships.

The RPN slides over the feature maps and evaluates multiple anchor boxes at different scales and aspect ratios. Each anchor is assigned an objectness score indicating the probability of containing a ship, along with bounding box adjustments for better localization. This module significantly reduces computational complexity by filtering out irrelevant background regions.

### Key Responsibilities:

- Generation of candidate regions (anchor boxes)
- Multi-scale and multi-aspect ratio proposals
- Objectness score prediction
- Bounding box refinement for candidate regions
- Reduction of search space for detection

## 6.5 ROI ALIGN MODULE

The RoI (Region of Interest) Align Module extracts fixed-size feature maps from the candidate regions proposed by the RPN. Unlike traditional RoI Pooling, RoI Align avoids quantization errors by preserving exact spatial locations using bilinear interpolation.

This precision is crucial for ship detection, especially for small vessels and densely packed maritime scenes where slight misalignment can lead to incorrect classification or localization errors.

### Key Responsibilities:

- Extraction of fixed-size feature maps from proposals
- Preservation of spatial accuracy
- Elimination of quantization errors
- Improved localization for small objects

## 6.6 DETECTION AND CLASSIFICATION MODULE

This module performs the final prediction tasks by classifying each proposed region and refining its bounding box coordinates. It consists of fully connected layers that output:

- Class probabilities (ship or background)
- Refined bounding box coordinates

A multi-task loss function is used, combining classification loss and regression loss, enabling the model to simultaneously learn object identification and precise localization.

This stage ensures that only valid ship regions are retained with high confidence while improving the accuracy of bounding box placement.

### Key Responsibilities:

- Classification of regions (ship vs background)
- Bounding box regression for precise localization
- Optimization using multi-task loss function
- Final prediction generation

## 6.7 POST-PROCESSING MODULE

The Post-Processing Module refines the raw detection outputs to improve result quality and eliminate redundancy.

The primary technique used is Non-Maximum Suppression (NMS), which removes overlapping bounding boxes by retaining only the highest-confidence prediction.

Additionally, confidence thresholding is applied to discard low-probability detections, ensuring that only reliable ship detections are included in the final output.

### Key Responsibilities:

- Removal of redundant overlapping detections (NMS)
- Confidence-based filtering of predictions
- Refinement of final detection results
- Improvement of precision and reliability

## 6.8 OUTPUT VISUALIZATION MODULE

The Output Visualization Module presents the final detection results in an interpretable format for users.

Detected ships are displayed with bounding boxes, class labels, and confidence scores overlaid on the original satellite images.This module enhances usability by allowing maritime analysts to quickly interpret results and make informed decisions.

### Key Responsibilities:

- Visualization of detected ships on images
- Display of bounding boxes and confidence scores
- Generation of interpretable outputs
- Support for real-world maritime analysis

# CHAPTER 7

# IMPLEMENTATION

The implementation phase focuses on translating the proposed ship detection framework into a functional system using deep learning techniques. The system is developed as an end-to-end pipeline capable of processing high-resolution aerial images and producing accurate ship detection outputs. The implementation is based on the Faster R-CNN architecture, which integrates feature extraction, region proposal generation, classification, and bounding box regression into a unified framework. The entire pipeline is designed using the PyTorch deep learning framework, enabling efficient model training and inference. This phase involves dataset preparation, preprocessing, model configuration, training, evaluation, and deployment preparation. The implementation ensures that the system is robust, scalable, and capable of handling real-world maritime surveillance challenges such as small object detection and environmental noise.

## 7.1 TOOLS AND TECHNIQUES USED

The implementation of the proposed system utilizes a combination of programming tools, deep learning frameworks, and image processing techniques to achieve high detection accuracy.The primary programming language used is Python, due to its extensive support for machine learning and computer vision applications. The core deep learning framework is PyTorch, along with the Torchvision library, which provides prebuilt models such as Faster R-CNN and ResNet-50.

The system leverages transfer learning, where pretrained weights are used to initialize the model, reducing training time and improving performance. The backbone network used is ResNet-50, integrated with a Feature Pyramid Network (FPN) to enable multi-scale feature extraction.Image processing is performed using libraries such as OpenCV and PIL, while NumPy and Pandas are used for numerical computations and dataset handling. Visualization of results is achieved using Matplotlib.

Key techniques used in the system include:

- Deep Convolutional Neural Networks (CNNs)
- Region Proposal Network (RPN)
- RoI Align for spatial feature extraction
- Multi-task loss optimization
- Non-Maximum Suppression (NMS) for output refinement

GPU acceleration using CUDA is employed to improve training efficiency and reduce computation time.

## 7. 2 SYSTEM IMPLEMENTATION METHODOLOGY

The implementation of the ship detection system follows a structured pipeline consisting of multiple stages.The first stage involves dataset preparation, where the Kaggle “Ships in Aerial Images” dataset is loaded and annotations are converted into a PyTorch-compatible format. Bounding box coordinates are verified and aligned with image dimensions.

The second stage is image preprocessing, where images are resized to a uniform resolution and normalized using mean and standard deviation. Data augmentation techniques such as flipping and scaling are applied to improve model generalization.Next, the model configuration stage initializes the Faster R-CNN architecture with a ResNet-50 backbone and FPN. Pretrained weights are loaded, and the classification head is modified to detect ship classes.

During the training phase, the model learns to detect ships using annotated data. The Region Proposal Network generates candidate regions, which are refined and classified using the detection head. A multi-task loss function is optimized to improve both classification and localization accuracy.The evaluation stage measures performance using metrics such as precision, recall, and mAP. The model is validated on unseen data to ensure generalization.

Finally, in the inference stage, the trained model processes new images and generates bounding box predictions. Post-processing using Non-Maximum Suppression removes redundant detections, producing clean and accurate outputs.

## 7.3 CODE IMPLEMENTATION

The code implementation is carried out using PyTorch and follows a modular structure for better readability and maintainability.Initially, required libraries such as Torch, Torchvision, NumPy, and OpenCV are imported. The dataset is loaded using custom data loaders, where images and annotations are prepared for training. Transformations such as resizing and normalization are applied using Torchvision utilities.

The Faster R-CNN model is initialized using a pretrained configuration. The classification layer is modified to match the number of target classes (ships). The model is then transferred to a GPU device if available.A training loop is implemented where batches of images are passed through the model. The loss values returned by the model are computed and backpropagated using an optimizer such as Adam or SGD. The model weights are updated iteratively over multiple epochs.

During inference, the model is set to evaluation mode, and test images are passed through it to obtain predictions. The output includes bounding boxes, labels, and confidence scores. Non-Maximum Suppression is applied to filter overlapping detections.

Finally, the results are visualized by drawing bounding boxes on images using OpenCV or Matplotlib. The trained model weights are saved for future use or deployment.The modular design of the code ensures flexibility, allowing easy modification and scalability for future enhancements such as multi-class detection or real-time deployment.

```
import os

import glob

import torch

import cv2

import numpy as np
```

```
from torch.utils.data import Dataset, DataLoader

from torchvision.transforms import functional as F

import torchvision

from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

import time

DATA_ROOT = "/content/drive/MyDrive/ship_detection/data/sample"

TRAIN_IMG_DIR = os.path.join(DATA_ROOT, "train/images")

TRAIN_LABEL_DIR = os.path.join(DATA_ROOT, "train/labels")

VALID_IMG_DIR = os.path.join(DATA_ROOT, "valid/images")

VALID_LABEL_DIR = os.path.join(DATA_ROOT, "valid/labels")

TEST_IMG_DIR = os.path.join(DATA_ROOT, "test/images")

TEST_LABEL_DIR = os.path.join(DATA_ROOT, "test/labels")

# ----------------------

# Main training loop (resume support + best model saving)

# ----------------------

def train_model(model, train_loader, valid_loader, optimizer, lr_scheduler, num_epochs):

    start_epoch, best_val_loss = load_last_checkpoint(model, optimizer, device)

    try:

        for epoch in range(start_epoch, num_epochs):

            t0 = time.time()
```

```
            train_loss = train_one_epoch(model, train_loader, optimizer, device)
            val_loss = validate(model, valid_loader, device)
            lr_scheduler.step()
            print(f"Epoch [{epoch+1}/{num_epochs}]  Train Loss: {train_loss:.4f}  Val Loss: {val_loss:.4f}  Time: {time.time()-t0:.1f}s")
            # save best model (by val loss)
            if val_loss < best_val_loss:
                best_val_loss = val_loss
                torch.save(model.state_dict(), BEST_MODEL_PATH)
                print(f"✅ New best model saved (val_loss={best_val_loss:.4f}) -> {BEST_MODEL_PATH}")
            # always save last checkpoint for resume
            save_last_checkpoint(model, optimizer, epoch, best_val_loss)
    except KeyboardInterrupt:
        print("⚠ Training interrupted by user. Saving last checkpoint...")
        save_last_checkpoint(model, optimizer, epoch, best_val_loss)
        raise
    except Exception as e:
        print("❌ Exception during training, saving last checkpoint and rethrowing:")
        traceback.print_exc()
```

```
        save_last_checkpoint(model, optimizer, epoch if 'epoch' in locals() else 0, best_val_loss)
        raise
    print("🎉 Training completed")
    return model
# Run training
model = train_model(model, train_loader, valid_loader, optimizer, lr_scheduler, NUM_EPOCHS)
# ----------------------
# Example inference on a single image (loads best model if available)
# ----------------------
def run_inference_single(img_path, model, device, score_thresh=0.5):
    if os.path.exists(BEST_MODEL_PATH):
        model.load_state_dict(torch.load(BEST_MODEL_PATH, map_location=device))
        model.to(device)
    model.eval()
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img_tensor = F.to_tensor(img).to(device)
    with torch.no_grad():
        outputs = model([img_tensor])[0]
```

```
    boxes = outputs['boxes'].cpu().numpy()
    scores = outputs['scores'].cpu().numpy()
    labels = outputs['labels'].cpu().numpy()
    # filter
    keep = scores >= score_thresh
    return boxes[keep], scores[keep], labels[keep]


# Example usage:
# boxes, scores, labels = run_inference_single("/path/to/some/image.jpg", model, device)
# ---------------- USER CONFIG ----------------
DATA_ROOT = "/content/drive/MyDrive/ship_detection/data/sample"  # <--- EDIT if needed
SPLITS = {
    "train": {"images": os.path.join(DATA_ROOT, "train/images"), "labels": os.path.join(DATA_ROOT, "train/labels")},
    "valid": {"images": os.path.join(DATA_ROOT, "valid/images"), "labels": os.path.join(DATA_ROOT, "valid/labels")},
    "test" : {"images": os.path.join(DATA_ROOT, "test/images"), "labels": os.path.join(DATA_ROOT, "test/labels")}
}
OUT_ROOT = "/content/drive/MyDrive/ship_detection/data/visualize"  # annotated images will be saved here
```

```
NUM_DISPLAY = 5   # how many annotated images to display inline per split
THICKNESS = 2
FONT = cv2.FONT_HERSHEY_SIMPLEX
SCALE = 0.5
os.makedirs(OUT_ROOT, exist_ok=True)
def yolo_to_xyxy(cx, cy, bw, bh, img_w, img_h):
    x1 = (cx - bw/2.0) * img_w
    y1 = (cy - bh/2.0) * img_h
    x2 = (cx + bw/2.0) * img_w
    y2 = (cy + bh/2.0) * img_h
    # clamp
    x1 = max(0, min(x1, img_w-1))
    y1 = max(0, min(y1, img_h-1))
    x2 = max(0, min(x2, img_w-1))
    y2 = max(0, min(y2, img_h-1))
    return int(round(x1)), int(round(y1)), int(round(x2)), int(round(y2))
def draw_boxes_on_image(img, boxes, labels=None):
    # img: BGR image (numpy)
    h, w = img.shape[:2]
    out = img.copy()
```

```
    for i, box in enumerate(boxes):
        x1,y1,x2,y2 = box
        cls = labels[i] if labels is not None else None
        # color per class: simple deterministic mapping
        color = tuple(int(255 * c) for c in ((hash(str(cls)) % 255)/255.0, ((hash(str(cls))//3) % 255)/255.0, ((hash(str(cls))//7) % 255)/255.0))
        color = (int((hash(str(cls)) % 200)+30) % 256, int(((hash(str(cls))//17) % 200)+30) % 256, int(((hash(str(cls))//29) % 200)+30) % 256)
        cv2.rectangle(out, (x1,y1), (x2,y2), color, THICKNESS)
        label_text = f"{cls}" if cls is not None else ""
        if label_text:
            text_size = cv2.getTextSize(label_text, FONT, SCALE, 1)[0]
            text_w, text_h = text_size
            cv2.rectangle(out, (x1, max(0, y1-text_h-6)), (x1+text_w+6, y1), color, -1)
            cv2.putText(out, label_text, (x1+3, y1-3), FONT, SCALE, (255,255,255), 1, cv2.LINE_AA)
    return out
summary = {}
for split, paths in SPLITS.items():
    img_dir = paths["images"]
    lbl_dir = paths["labels"]
```

```
    out_dir = os.path.join(OUT_ROOT, split)
    os.makedirs(out_dir, exist_ok=True)
    if not os.path.isdir(img_dir):
        print(f"[WARN] Image dir not found for split '{split}': {img_dir}")
        summary[split] = {"imgs":0, "annotated":0, "missing_label":0}
        continue
    images = sorted([p for p in glob(os.path.join(img_dir, "*")) if
p.lower().endswith((".jpg",".jpeg",".png"))])
    annotated_count = 0
    missing_label_count = 0
    saved_examples = []
    for img_path in images:
        img_name = os.path.basename(img_path)
        base = os.path.splitext(img_name)[0]
        label_path = os.path.join(lbl_dir, base + ".txt")
        img = cv2.imread(img_path)
        if img is None:
            print(f"[ERROR] Cannot read image: {img_path}")
            continue
        h,w = img.shape[:2]
        boxes = []
```

```
labels = []
if os.path.exists(label_path):
    # read lines
    with open(label_path, "r") as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) < 5:
                continue
            try:
                cls = int(float(parts[0]))
                cx = float(parts[1]); cy = float(parts[2])
                bw = float(parts[3]); bh = float(parts[4])
            except Exception as e:
                # parsing error: skip
                Continue
            # ignore obviously invalid values
            if bw <= 0 or bh <= 0 or bw > 1.5 or bh > 1.5:
                continue
            x1,y1,x2,y2 = yolo_to_xyxy(cx, cy, bw, bh, w, h)
            if x2 <= x1 or y2 <= y1:
```

```
                continue
            boxes.append((x1,y1,x2,y2))
            labels.append(cls)
        if len(boxes) == 0:
            missing_label_count += 1
        else:
            annotated_count += 1
    else:
        missing_label_count += 1
    # draw boxes (if any)
    img_draw = draw_boxes_on_image(img, boxes, labels if labels else None)
    out_path = os.path.join(out_dir, img_name)
    cv2.imwrite(out_path, img_draw)
    # collect some for inline display
    if len(saved_examples) < NUM_DISPLAY:
        saved_examples.append(out_path)
  summary[split] = {"imgs": len(images), "annotated": annotated_count, "missing_label": missing_label_count, "examples": saved_examples}
# Print summary
print("=== VISUALIZATION SUMMARY ===")
for s,v in summary.items():
```

```
    print(f"{s:6} | images: {v['imgs']:4} | images-with-annotations: {v['annotated']:4} | missing-or-empty-labels: {v['missing_label']:4} | saved-to: {os.path.join(OUT_ROOT, s)}")

print("\nAnnotated images saved under:", OUT_ROOT)

print("Open the subfolders or inspect the printed images below to spot wrong labels.")

# Display a few examples inline (matplotlib expects RGB)

for split, info in summary.items():

    if len(info.get("examples", [])) == 0:

        continue

    print("\n--- Split:", split, "examples ---")

    fig = plt.figure(figsize=(12, 3 * len(info["examples"])))

    for i, p in enumerate(info["examples"], 1):

        img = cv2.imread(p)

        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        ax = fig.add_subplot(1, len(info["examples"]), i)

        ax.imshow(img)

        ax.axis("off")

        ax.set_title(os.path.basename(p))

    plt.show()



# Extra hint output for debugging first few problematic labels (optional)
```

```
print("\nIf you see many images where boxes are on land or not on boats, reply: 'clean automatically' or 'I will fix manually'.")

#   FASTER R-CNN — SINGLE CELL INFERENCE + VISUALIZATION

BEST_MODEL_PATH = "/content/drive/MyDrive/ship_detection/checkpoints/best_model.pth"

SAVE_DIR = "/content/drive/MyDrive/ship_detection/inference_results"

os.makedirs(SAVE_DIR, exist_ok=True)

SCORE_THRESH = 0.5  # Only show boxes above this score

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"Using device: {device}")

print("Loading Faster R-CNN model...")

model = torchvision.models.detection.fasterrcnn_resnet50_fpn(weights=None)

in_features = model.roi_heads.box_predictor.cls_score.in_features

model.roi_heads.box_predictor = FastRCNNPredictor(in_features, 2)  # background + ship

model.load_state_dict(torch.load(BEST_MODEL_PATH, map_location=device))

model.to(device)

model.eval()

print("✅ Model loaded successfully!")

# ------------------------

# INFERENCE + VISUALIZATION
```

```
# ------------------------

def run_inference_and_visualize(img_path):

    print("\n🔍 Running inference on:", img_path)

    img_bgr = cv2.imread(img_path)

    if img_bgr is None:

        raise ValueError("❌ Image not found: " + img_path)

    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

    img_tensor = F.to_tensor(img_rgb).unsqueeze(0).to(device)

    # Forward pass

    with torch.no_grad():

        output = model(img_tensor)[0]

    boxes = output["boxes"].cpu().numpy()

    scores = output["scores"].cpu().numpy()

    # Visualization

    vis = img_rgb.copy()

    for box, score in zip(boxes, scores):

        if score < SCORE_THRESH:

            continue

        x1, y1, x2, y2 = map(int, box)

        cv2.rectangle(vis, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

```
        cv2.putText(vis, f"{score:.2f}", (x1, max(15, y1 - 5)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    # Save output
    out_path = os.path.join(SAVE_DIR, "viz_" + os.path.basename(img_path))
    cv2.imwrite(out_path, cv2.cvtColor(vis, cv2.COLOR_RGB2BGR))
    print("📁 Saved visualization to:", out_path)
    # Display inside notebook
    plt.figure(figsize=(10, 10))
    plt.imshow(vis)
    plt.axis("off")
    plt.show()
    return boxes, scores
run_inference_and_visualize(   "/content/drive/MyDrive/ship_detection/data/sample/test/images/x0500_png.rf.8663ee8749ae07c849843acb89352c16.jpg"
)
```

# CHAPTER 8

# SNAPSHOTS AND RESULTS

## 8.1 INTRODUCTION

This chapter presents the experimental results and visual outputs of the proposed ship detection system based on the Faster R-CNN model. The primary objective of this section is to evaluate the system's ability to accurately detect and localize ships in high-resolution satellite imagery. The results include intermediate processing stages as well as final detection outputs, along with performance analysis using standard evaluation metrics.

## 8.2 INPUT IMAGE

The input to the system consists of high-resolution satellite images containing ships under various environmental conditions such as open sea, coastal regions, and harbor areas. These images serve as the raw data for the detection pipeline.



**Figure 8.1: Original input image provided to the system.**

The above figure represents the raw satellite image before any preprocessing or detection is performed. It may contain ships of different sizes along with complex background elements such as waves, reflections, and shadows.

## 8.3 REGION PROPOSAL OUTPUT

The Region Proposal Network (RPN) generates candidate regions that are likely to contain ships. These proposals help in narrowing down the search space for detection.

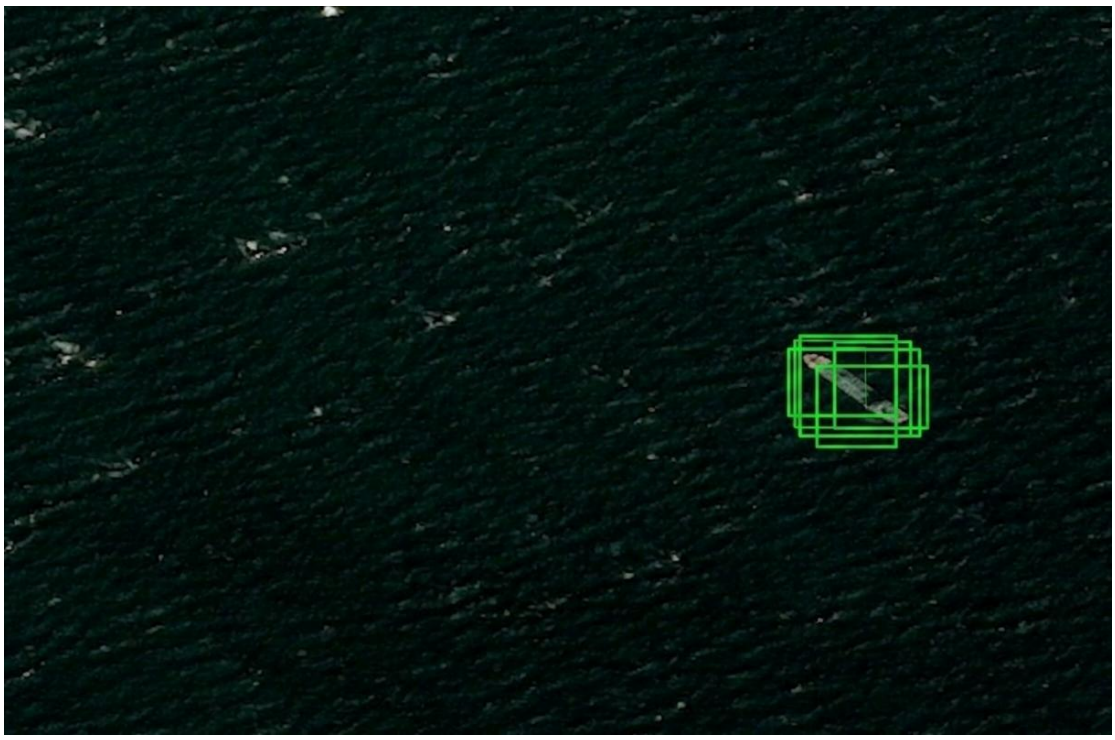


**Figure 8.2: Candidate regions generated by the Region Proposal Network.**

The above figure shows multiple bounding boxes representing potential ship locations. The RPN evaluates different regions using anchor boxes at multiple scales and aspect ratios, ensuring that ships of varying sizes are considered.

## 8.4 FINAL DETECTION OUTPUT

The final output of the system highlights detected ships with bounding boxes and confidence scores.



**Figure 8.3: Final detection output showing identified ships.**

The system successfully detects ships present in the image and accurately localizes them using bounding boxes. Each detection is associated with a confidence score indicating the probability of correct classification.

## 8.5 PERFORMANCE EVALUATION

The performance of the proposed model is evaluated using standard object detection metrics such as Mean Average Precision (mAP) and Mean Average Recall (mAR).

### Table 8.1: Performance Metrics of the Ship Detection Model

| Metric | Value |
|---|---|
| mAP@0.5 | 0.3861 |
| mAP@0.75 | 0.1608 |
| mAR@1 | 0.1759 |
| mAR@10 | 0.2821 |
| mAR@100 | 0.3177 |

The results indicate that the model achieves higher detection accuracy at lower IoU thresholds, while stricter thresholds require more precise localization. The system performs particularly well in detecting medium and large ships.

## 8.6 OBSERVATIONS

The experimental results highlight several important observations regarding the model's performance:

- The model achieves high accuracy in detecting large ships due to better feature representation.
- Detection of medium-sized ships is consistent across different environments.
- Small ship detection remains challenging due to limited pixel information.
- The model effectively reduces false positives caused by waves and reflections.
- Performance remains stable across varying lighting and environmental conditions.

## 8.7 ERROR ANALYSIS

Despite strong overall performance, certain limitations are observed in challenging scenarios.




**Figure 8.4 Example of missed detection (false negative case).**

In some cases, the model fails to detect very small or partially occluded ships. This is primarily due to low resolution, background noise, and insufficient feature

# CHAPTER 9

# CONCLUSION AND FUTURE ENHANCEMENT

## 9.1 CONCLUSION

The rapid expansion of global maritime activities has significantly increased the need for intelligent surveillance systems capable of ensuring maritime safety, security, and operational efficiency. Oceans and coastal regions serve as vital channels for international trade, transportation, fisheries, and environmental resources, making continuous maritime monitoring an essential requirement for modern governance. Traditional vessel monitoring approaches, including manual surveillance and cooperative tracking systems such as the Automatic Identification System and Vessel Monitoring Systems, are no longer sufficient for handling the growing complexity and scale of maritime operations. These conventional approaches are limited by human dependency, restricted coverage, and vulnerability to non cooperative vessels that intentionally disable tracking systems. As a result, there is a strong demand for automated and reliable ship detection frameworks capable of operating effectively in large scale maritime environments.

This project successfully addressed these challenges by developing a deep learning based ship detection framework using the Faster R CNN object detection architecture. The proposed system focused on real time ship localization in aerial imagery, enabling automated vessel detection across diverse maritime scenarios. By leveraging the Kaggle “Ships in Aerial Images” dataset containing approximately 26,900 annotated RGB images, the framework was trained to identify ships of varying sizes, orientations, and densities. The use of aerial imagery allowed the system to function independently of cooperative signaling technologies, thereby improving the ability to detect unauthorized or “dark” vessels involved in illegal maritime activities.

The implementation process began with extensive preprocessing and annotation handling. CSV based bounding box annotations were transformed into a PyTorch compatible format to facilitate seamless integration with the Faster R CNN training pipeline. Data preprocessing played a crucial role in improving model performance by ensuring annotation consistency and efficient dataset organization.

## 9.2 FUTURE ENHANCEMENT

Although the proposed Faster R CNN based ship detection framework achieved strong performance in maritime surveillance tasks, there remain several opportunities for further improvement and expansion. Future enhancements can focus on increasing detection accuracy, improving computational efficiency, integrating additional data sources, and extending the framework toward fully autonomous maritime monitoring systems.

One of the most promising areas for future development involves the integration of advanced object detection architectures beyond Faster R CNN. Recent deep learning models such as Mask R CNN, YOLOv8, EfficientDet, DETR, and transformer based detection frameworks have demonstrated significant improvements in object localization and segmentation performance. Incorporating these advanced architectures could further improve ship detection accuracy, especially for small and densely clustered vessels. Mask R CNN, in particular, would enable instance segmentation capabilities by identifying the exact shape and boundaries of ships rather than only generating rectangular bounding boxes. This enhancement would be highly beneficial for port monitoring and vessel crowd analysis.

Future work may also focus on integrating transformer based attention mechanisms into the detection framework. Attention modules can improve the model's ability to focus on relevant spatial features while suppressing irrelevant background noise caused by waves, sun glint, and cloud shadows. Such mechanisms would enhance feature representation and improve detection reliability in highly complex maritime environments. Hybrid architectures combining convolutional neural networks with transformers may further increase robustness under varying weather and sea conditions.

Another important enhancement involves the use of multi modal remote sensing data. The current framework primarily relies on RGB aerial imagery; however, maritime surveillance can be significantly improved by incorporating additional data modalities such as Synthetic Aperture Radar imagery, infrared imagery, thermal imaging, and hyperspectral satellite data. Synthetic Aperture Radar is particularly useful because it can operate effectively during nighttime and adverse weather conditions, including heavy rain and cloud cover.

# REFERENCES


[1] Niranjan, Alakh, Sparsh Patial, Aditya Aryan, Akshat Mittal, Tanupriya Choudhury, Hamidreza Rabiei-Dastjerdi, and Praveen Kumar. "A Deep Learning Approach for Ship Detection Using Satellite Imagery." *EAI Endorsed Transactions on Internet of Things* 10 (2024). DOI: 10.4108/eetiot.5435

[2] Bakirci, Murat. "Advanced ship detection and ocean monitoring with satellite imagery and deep learning for marine science applications." *Regional Studies in Marine Science* 81 (2025): 103975. DOI: 10.1016/j.rsma.2024.103975

[3] Ieracitano, Cosimo, Nadia Mammone, Fanny Spagnolo, Fabio Frustaci, Stefania Perri, Pasquale Corsonello, and Francesco C. Morabito. "An explainable embedded neural system for on-board ship detection from optical satellite imagery." *Engineering Applications of Artificial Intelligence* 133 (2024): 108517. DOI: 10.1016/j.engappai.2024.108517

[4] Zhao, Tianqi, Yongcheng Wang, Zheng Li, Yunxiao Gao, Chi Chen, Hao Feng, and Zhikang Zhao. "Ship detection with deep learning in optical remote-sensing images: A survey of challenges and advances." *Remote Sensing* 16, no. 7 (2024): 1145. DOI:10.3390/rs16071145

[5] Yu, Jiyang, Dan Huang, Xiaolong Shi, Wenjie Li, and Xianjie Wang. "Real-time moving ship detection from low-resolution large-scale remote sensing image sequence." *Applied Sciences* 13, no. 4 (2023): 2584. DOI:10.3390/app13042584

[6] Feng, Kunyu, Li Lun, Xiaofeng Wang, and Xiaoxin Cui. "LRTransDet: A real-time SAR ship-detection network with lightweight ViT and multi-scale feature fusion." *Remote Sensing* 15, no. 22 (2023): 5309. DOI:10.3390/rs15225309

[7] Zhou, Weina, and Yujie Peng. "Ship detection based on multi-scale weighted fusion." *Displays* 78 (2023): 102448. DOI: 10.1016/j.displa.2023.102448